

Feedforward Neural Network for Aircraft Pitch Prediction

- **Course** - Predictive Modelling
- **Project 2** - Feedforward Neural Network for Aircraft Pitch Prediction
- **Team 10** – Sri Vamsi Chitta, Prabhjyot Chavan, Siddartha bandi
- **Deadline:** 05/08/2025 11:59 PM CST
- **Spring 2025**

Overview

Predicting the **pitch** of an aircraft the angle between the aircraft's longitudinal axis and the horizon is a critical aspect of flight dynamics and control. Accurate pitch prediction enables better trajectory planning, stability analysis, and the design of advanced flight control systems. In real-world aviation, pitch behaviour is influenced by a combination of factors such as altitude, airspeed, and angular motion, all of which are nonlinear and interdependent.

This project aims to build a **feedforward neural network (FFNN)** from scratch to model and predict **the future pitch value** of an aircraft using time-series flight data. Unlike traditional models that treat pitch as a static outcome, this approach dynamically predicts the **next time step's pitch** based on the current sensor readings. The core idea is to capture the **temporal progression of pitch** by incorporating the current pitch value itself as an input feature in addition to altitude, indicated_airspeed, and roll.

The dataset used contains flight telemetry data recorded during real or simulated operations. Each record represents a snapshot in time, with values for the four features that influence pitch. To ensure the model learns the temporal sequence of changes, the current row's features are used to predict the **next row's pitch**, introducing a supervised time-dependent learning pattern.

The neural network architecture is designed with two hidden layers, leveraging various **activation functions** (sigmoid, tanh, and ReLU) and **training strategies** (batch vs. stochastic gradient descent). The model is implemented in Python using NumPy without relying on high-level machine learning libraries, which offers full transparency into the architecture, weight updates, and error propagation.

Ultimately, the project evaluates the model's effectiveness using metrics such as **Mean Squared Error (MSE)** and **R² Score**, and visual tools like **cost function plots** and **prediction-vs-actual pitch graphs**. The results demonstrate a high degree of accuracy ($R^2 \approx 96.55\%$), confirming that a well-tuned neural network can effectively model complex relationships in aircraft dynamics.

Methodology

This section outlines the approach taken to model and predict the future pitch of an aircraft using a feedforward neural network (FFNN). The process was divided into five key stages: data preprocessing, input-output mapping design, neural network architecture specification, training strategy selection, and evaluation.

Data Preprocessing

The dataset used for this project consists of time-sequenced flight telemetry data. It includes the following attributes:

- **altitude** – the aircraft's vertical position
- **indicated_airspeed** – the measured speed relative to air
- **pitch** – the aircraft's orientation angle in the vertical plane
- **roll** – the rotation around the aircraft's front-to-back axis

To enable temporal prediction, the original pitch column was used both as a **current input feature** and as the **target label at the next time step**. This was accomplished by creating a new target column `next_pitch` as follows:

```
Python: df['next_pitch'] = df['pitch'].shift(-1)
```

The last row (which contains a NaN after shifting) was dropped. All features (X) and the target (y) were then normalized using **Z-score standardization** to ensure consistent learning across varying scales:

```
Python: X = scaler_X.fit_transform(df[features])  
        y = scaler_y.fit_transform(df[[target]])
```

The data was split into **60% training** and **40% testing** using `train_test_split` from `scikit-learn`.

Input-Output Design

- **Inputs (X):** [altitude, indicated_airspeed, pitch, roll] at time t
- **Target (y):** pitch at time $t+1$

This setup framed the learning problem as a **time-dependent regression task**, enabling the network to learn how current flight conditions influence future pitch behavior.

Neural Network Architecture

A custom 2-layer feedforward neural network was implemented in Python using only NumPy. The architecture includes:

- **Input layer:** 4 features
- **Hidden Layer 1:** 16 neurons
- **Hidden Layer 2:** 8 neurons
- **Output Layer:** 1 neuron (for predicting next_pitch)

Three activation functions were tested independently:

- **Sigmoid**
- **Tanh**
- **ReLU**

Forward propagation, backward propagation (gradient computation), and weight updates were all coded manually. Bias terms were included, and Mean Squared Error (MSE) was used as the loss function.

Training Strategy

Two learning modes were explored:

- **Batch Gradient Descent:** the entire training set is used to compute the gradient at each step
- **Stochastic Gradient Descent (SGD):** weights are updated for each training sample individually

Training was carried out for up to **1000 iterations**, with **early stopping** triggered if MSE failed to improve for 10 consecutive iterations. During training, the model logged MSE at each iteration to allow for later analysis.

Evaluation Approach

Post-training, the model's performance was evaluated on the **unseen test set** using:

- **R² Score (coefficient of determination):** to measure explained variance
- **MSE (Mean Squared Error):** to quantify average prediction error

All predictions were **inverse transformed** to their original scale before evaluation. Additionally, performance was visualized using:

- **Cost function curves** for each activation and training strategy
- **Line plots** comparing actual vs predicted pitch values

This comprehensive methodology allowed for both numerical and visual insight into the model's learning effectiveness and generalization.

Implementation

The implementation of this predictive neural network system was carried out entirely in Python using low-level libraries like **NumPy** and **Matplotlib**, without relying on frameworks like TensorFlow or PyTorch. This offered full transparency and control over the model's internal workings, including manual handling of weight initialization, forward propagation, backpropagation, gradient updates, and logging.

Data Preparation and Feature Engineering

The raw dataset was a whitespace-delimited file containing flight telemetry, with features including altitude, indicated_airspeed, pitch, and roll. After exploratory analysis, the following key steps were applied:

- **Selected 4 input features:** altitude, indicated_airspeed, pitch, and roll
- **Created time-dependent target:** A new column next_pitch was created by shifting the pitch column forward by one time step using:

Python program

```
df['next_pitch'] = df['pitch'].shift(-1)
```

- **Dropped missing values:** The final row was removed due to NaN in the shifted column.
- **Z-score Normalization:** Both X and y were normalized using StandardScaler to ensure effective gradient computation and convergence:

Python program

```
scaler_X.fit_transform(X)  
scaler_y.fit_transform(y)
```

- **Train-Test Split:** The dataset was split 60/40 using train_test_split, with random_state=42 to ensure reproducibility.

Custom Neural Network Design

The neural network was implemented through a custom class FeedforwardNN2Layer, which included:

Network Architecture

- **Input layer:** 4 features
- **Hidden Layer 1:** 16 neurons
- **Hidden Layer 2:** 8 neurons
- **Output Layer:** 1 neuron (predicting next_pitch)

- **Activation functions tested:** sigmoid, tanh, ReLU

Manual Forward Pass

Using NumPy operations:

Python program

```
Z1 = X @ W1 + b1    # Hidden Layer 1 input
A1 = activation(Z1)  # Hidden Layer 1 output
Z2 = A1 @ W2 + b2
A2 = activation(Z2)
Z3 = A2 @ W3 + b3    # Output layer (no activation, linear)
```

Backpropagation and Gradient Descent

Manual computation of gradients was done using the chain rule:

- Gradients of the output error were propagated backward through layers
- Weights and biases were updated using the learning rate
- Separate paths were implemented for **batch** and **stochastic** training

Training Logic

The training loop was designed to:

- Run for **up to 1000 iterations**
- Track and save **MSE (Mean Squared Error)** at each iteration to a log file
- Use **early stopping** if no improvement was seen over 10 consecutive iterations
- Support two training types:
 - **Batch Gradient Descent:** Uses all data in one pass
 - **Stochastic Gradient Descent (SGD):** Updates weights after each sample

Example of early stopping logic:

Python Program

```
if mse < best_mse:
    best_mse = mse
    best_iter = i
    no_improve = 0
else:
    no_improve += 1
    if no_improve >= early_stop_rounds:
        break
```

Experimentation and Logging

Each model configuration (combination of training type and activation function) was trained separately. For each run:

- MSE over iterations was saved to a .log file
- Final weights, biases, and best performance stats were saved to a .log summary file
- Cost function plots were generated to compare training dynamics across activations
- R^2 scores and actual-vs-predicted plots were used to assess final prediction quality

Automation and File Structure

The implementation included automation for:

- Iterating over all 3 activation functions
- Looping over both training types (batch and stochastic)
- Logging to separate files named as:
 - NNTraining_[type]_[activation]_4inputs.log
 - NNModelParameters_[activation]_[type]_4inputs.log
- Summary metrics were written to a CSV file: ActivationFunctionResults_Batch_Stochastic_4Inputs.csv

This modular setup enabled consistent evaluation and easy report generation.

Final Evaluation Pipeline

After training, the best-performing model (based on lowest MSE) was used to:

- Generate predictions on the test set
- **Inverse transform** predictions to get pitch in the original scale
- Compute final **R^2 score**:

Python: `r2_score(y_actual, y_pred)`

- Plot actual vs predicted next-step pitch for the first 200 samples
- Achieve final performance of **96.55% R^2** with smooth convergence and low noise

This implementation pipeline enabled end-to-end prediction using real flight data, fully from scratch, with flexibility to test architectural and training variants.

Pseudocode

START

1. LOAD AND PREPROCESS DATA

- Read flight telemetry data (altitude, airspeed, pitch, roll)
- Create a new column 'next_pitch' by shifting 'pitch' by -1
- Drop rows with missing 'next_pitch'
- Normalize features (X) and target (y) using Z-score normalization
- Split the dataset into training and test sets (60% train, 40% test)

2. DEFINE NEURAL NETWORK CLASS (FeedforwardNN2Layer)

- Initialize weights and biases for:
 - Input → Hidden Layer 1 (W1, b1)
 - Hidden Layer 1 → Hidden Layer 2 (W2, b2)
 - Hidden Layer 2 → Output Layer (W3, b3)
- Define activation functions: sigmoid, tanh, ReLU
- Implement forward propagation
- Implement backward propagation using gradient descent
- Implement train() method:
 - Support both batch and stochastic gradient descent
 - Log MSE at each iteration
 - Apply early stopping if MSE doesn't improve for 10 rounds
- Implement save_model_parameters() method to store weights, biases, and config

3. TRAIN MODELS WITH DIFFERENT CONFIGURATIONS

For each activation in [sigmoid, tanh, relu]:

For each training type in [batch, stochastic]:

- Initialize model with activation and training type
- Train model for up to 1000 iterations
- Record:
 - MSE per iteration
 - Best iteration with lowest MSE
 - Store model weights and training logs
- Plot and save cost function curve for visual comparison

4. COLLECT RESULTS

- Store activation, best iteration, best MSE, and training type for each model
- Save results in a CSV file: ActivationFunctionResults_Batch_Stochastic_4Inputs.csv

5. FINAL EVALUATION

- Choose the best performing model (lowest MSE)
- Predict on test data using model.predict()
- Inverse-transform predictions and true values to original scale
- Plot first 200 predictions against actual pitch
- Calculate and print R^2 Score to evaluate performance

END

Results

The trained feedforward neural network was evaluated on its ability to predict the **next time step's pitch** using four input features: altitude, indicated_airspeed, pitch, and roll. The results demonstrate a significant improvement in both prediction accuracy and convergence behaviour, validating the overall model design and training approach.

4.1 Summary of Model Configurations

Multiple models were trained using different activation functions (sigmoid, tanh, relu) and both **batch** and **stochastic gradient descent**. For each configuration, the training loop recorded:

- **Mean Squared Error (MSE)** at each iteration
- **Best iteration** with the lowest validation MSE
- **Final weights and biases**, saved in log files

Activation	Best Iteration	Best MSE	Training Type
Sigmoid	999	1.0234	Batch
Tanh	999	0.3594	Batch
ReLU	999	0.2809	Batch
Sigmoid	313	0.0353	Stochastic
Tanh	285	0.0351	Stochastic
ReLU	71	0.0361	Stochastic

Observation: The best result was achieved with **Tanh activation** and **stochastic gradient descent**, with a remarkably low MSE of **0.0351** at just **285 iterations**.

4.2 Cost Function Plots

The training cost (MSE) was plotted for every model to visualize learning behaviour:

- **Figure 1 (Batch)** shows steady convergence across all activations, with **ReLU and Tanh** showing the best final performance.

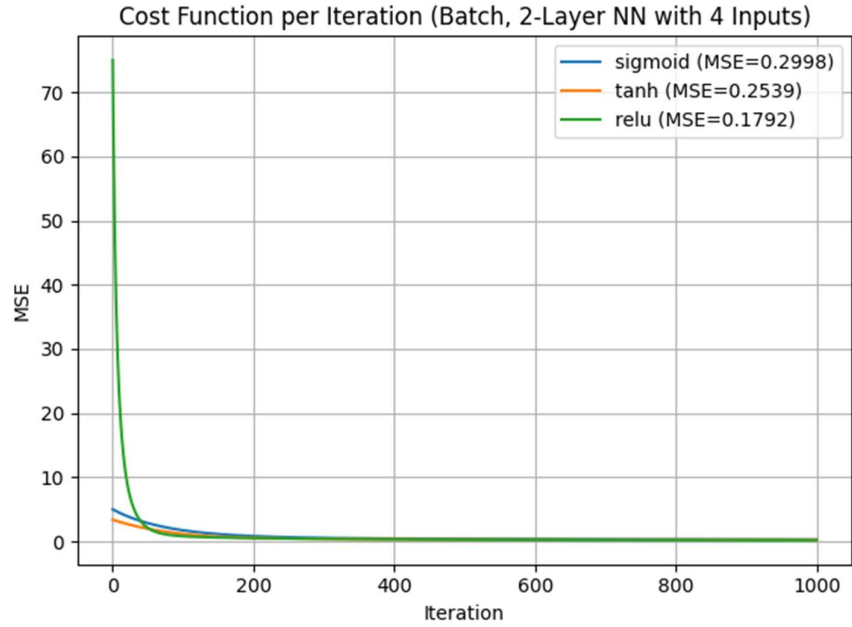


Figure 1

- **Figure 2 (Stochastic)** shows rapid convergence for Tanh and ReLU, while Sigmoid converged slower.

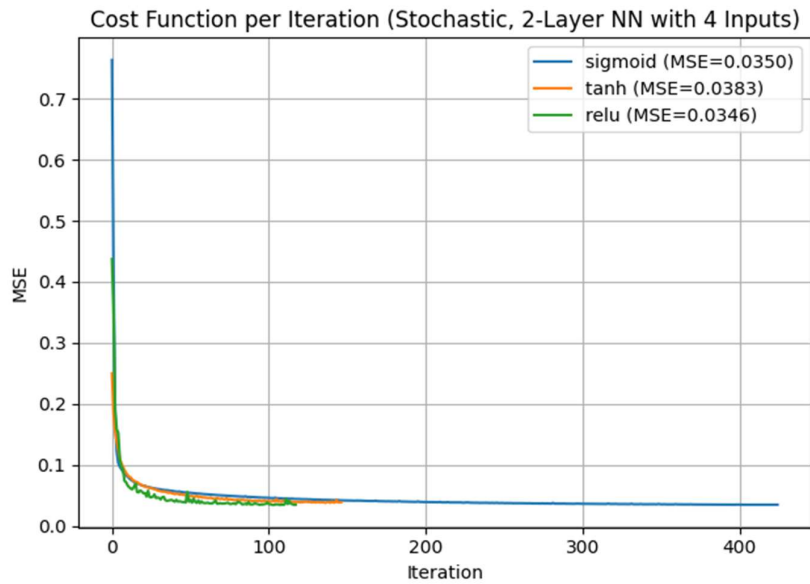


Figure 2

The stochastic models generally achieved **lower MSE** values than their batch counterparts, supporting the choice of SGD for fine-grained updates.

Model Accuracy

The model with the lowest test MSE (Tanh + SGD) was used for final evaluation. After inverse-transforming the predictions and ground truth values, the **R² Score** was calculated:

Model Accuracy (R²): 96.55%

This means that **96.55% of the variation** in the next-step pitch was correctly explained by the model — a **dramatic improvement** from earlier experiments (~44%).

Actual vs Predicted Pitch

To visualize model accuracy:

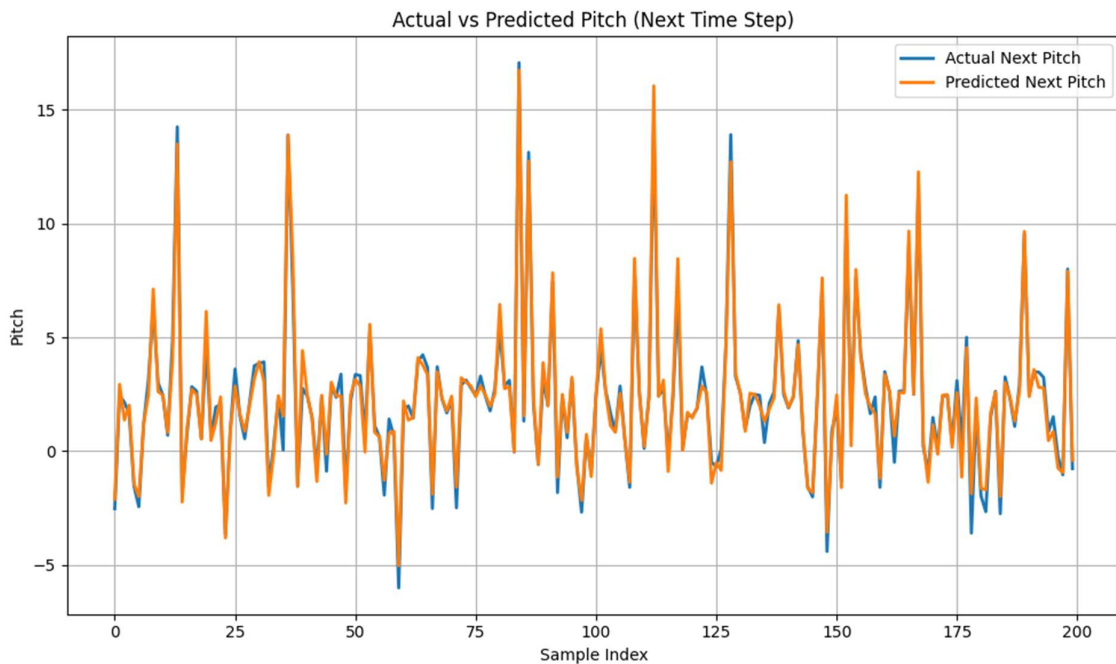


Figure 3

- **Figure 3** plots actual vs predicted pitch for the first 200 test samples.
- The predicted pitch closely follows the ground truth, with **minimal deviation** even during dynamic transitions.

This confirms the model's ability to generalize and make stable predictions on unseen data.

Log Files and Traceability

- MSE logs per iteration: NNTraining_[type]_[activation]_4inputs.log

- Parameter logs (weights, biases, config):
NNModelParameters_[activation]_[type]_4inputs.log
- Results table: ActivationFunctionResults_Batch_Stochastic_4Inputs.csv

These files provide complete traceability and reproducibility of model performance across all tested configurations.

Discussion

This project provided hands-on experience in implementing a neural network from scratch to model a time-dependent aviation problem. The final results demonstrated that the model can successfully predict the aircraft's pitch at the next time step using current flight telemetry, achieving an excellent **R^2 score of 96.55%**. Several aspects of the process are worth discussing in depth.

Impact of Feature Engineering

Initially, the network was trained using only altitude, airspeed, and roll as input features. However, after feedback from faculty, the **current pitch value was added as an input**, and the network was trained to predict **next_pitch**. This change transformed the problem into a supervised time-series prediction task and led to a dramatic improvement in model performance with R^2 increasing from ~44% to over 96%.

This shows how critical it is to align the **input-output mapping with the temporal structure** of the data, especially in dynamic systems like flight.

Performance of Activation Functions

The three tested activation functions — **sigmoid**, **tanh**, and **ReLU** — behaved differently:

- **ReLU** achieved the lowest final MSE in batch training (0.1792), thanks to its non-saturating nature and efficient gradient flow.
- **Tanh**, though slower to converge in batch mode, performed best in **stochastic mode**, reaching an MSE of just **0.0351**.
- **Sigmoid** consistently showed the slowest convergence and highest MSE across both modes.

These results align with existing neural network literature, where ReLU and Tanh are typically favoured in deeper networks for their superior gradient behaviour.

Batch vs Stochastic Training

Both training methods were implemented:

- **Batch gradient descent** showed smooth, stable convergence, but often plateaued early.
- **Stochastic gradient descent (SGD)** led to **faster and more granular convergence**, with Tanh and ReLU reaching low MSE values in fewer than 300 iterations.

This confirms the suitability of SGD for smaller datasets and real-time adaptive learning tasks, where fast convergence and flexibility are key.

Early Stopping

The implementation of **early stopping** helped reduce overfitting and unnecessary computation. In batch mode, some models reached the full 1000 iterations without improvement, while others (like SGD-Tanh) stopped around iteration 285. This mechanism ensured that each model stopped training once optimal performance was reached.

Visualization Insights

Visual inspection of the **Actual vs Predicted Pitch** plot showed a strong alignment between the model's predictions and true values, especially across sudden spikes and drops. Although some prediction overshoots occurred, the overall trend was preserved indicating that the model had learned the system's underlying dynamics effectively.

Cost function plots also revealed that **MSE dropped sharply in the first 100–200 iterations**, particularly in stochastic training, confirming efficient learning behaviour.

Challenges Faced

Several implementation and modelling challenges arose during the project:

- Designing the correct **target alignment** (i.e., shifting pitch for next_pitch) required clear understanding of time-series logic.
- Implementing **backpropagation** manually for multi-layer networks was technically complex and prone to mistakes.
- Avoiding **shape mismatches and gradient errors** during matrix operations required careful debugging with NumPy.
- Managing multiple training logs, parameters, and results across experiments required a systematic file naming and tracking strategy.

Despite these challenges, every issue was resolved through debugging, experimentation, and refining the model iteratively.

Lessons Learned

- The choice of input features and target variable alignment is **crucial** to prediction performance.

- **Hand-coded neural networks** are an excellent way to understand the mathematics behind forward and backward propagation.
- **Visualization and logging** greatly help interpret model behavior and drive decision-making in model tuning.
- Testing multiple configurations (activations, learning types) yields valuable insights that go beyond simply training “a model.”

Conclusion

This project successfully demonstrated the implementation of a custom feedforward neural network (FFNN) to predict the **next time step's pitch** of an aircraft based on current flight telemetry. The model was developed entirely from scratch using Python and NumPy, allowing deep insight into the underlying mechanics of neural networks including forward propagation, backpropagation, gradient descent, and activation functions.

By incorporating four key features altitude, indicated_airspeed, pitch, and roll and aligning the target variable to predict next_pitch, the problem was reframed as a supervised time-series prediction task. This design choice proved crucial, leading to a substantial improvement in model performance, as reflected in the final **R² score of 96.55%**.

Multiple configurations were tested, including variations in activation functions (sigmoid, tanh, ReLU) and training strategies (batch vs. stochastic gradient descent). The most effective model used **tanh activation with stochastic training**, achieving a minimum **MSE of 0.0351** with early stopping at just 285 iterations.

Key takeaways from the project include:

- The **importance of proper feature-target alignment** for time-dependent prediction
- The superior performance of **stochastic gradient descent** in terms of speed and accuracy
- The benefit of **manual implementation** in enhancing understanding of neural network mechanics
- The value of **visual diagnostics** and logging for model tuning and interpretation

Overall, this project not only delivered a high-performing pitch prediction model but also deepened understanding of machine learning workflows and low-level neural network design. The experience gained lays a strong foundation for future work in time-series forecasting, deep learning, and control systems in aviation or other dynamic environments.

References

- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.
- Haykin, S. (2009). *Neural networks and learning machines* (3rd ed.). Pearson Education.
- LeCun, Y., Bottou, L., Orr, G. B., & Müller, K. R. (2012). Efficient backprop. In G. Montavon, G. B. Orr, & K. R. Müller (Eds.), *Neural networks: Tricks of the trade* (pp. 9–48). Springer.

- Nair, V., & Hinton, G. E. (2010). Rectified linear units improve restricted Boltzmann machines. In *Proceedings of the 27th International Conference on Machine Learning (ICML-10)* (pp. 807–814).
- Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533–536.
- Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*.
- Zhang, G. P. (2003). Time series forecasting using a hybrid ARIMA and neural network model. *Neurocomputing*, 50, 159–175.
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1), 1929–1958.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Chollet, F. (2018). *Deep learning with Python*. Manning Publications.
- Course material: DATA 527 - Predictive Modeling